YU CHUAN OH

AI / ML Engineer | LLM Systems | Computer Vision | Applied Machine Learning in Python

Berlin, Germany | Authorized to work in Germany, no sponsorship required | Open to Europe-remote roles (CET)
+49 176 77628402 | ohyuchuan@gmail.com | linkedin.com/in/yu-chuan-oh-169a3419b | github.com/yc82628 | yc82628.github.io

SUMMARY

AI/ML engineer who builds machine learning and LLM systems that fail safely instead of confidently: input quality gates, calibrated models that abstain on borderline cases, leakage-proof evaluation, sandboxed evidence pipelines and multi-provider LLM fallback. Designed and built a diabetic retinopathy screening system end to end on my own (PyTorch, FastAPI), from patient-grouped data splits and calibration to an audited grading service. Top 10 at the Daytona AI Builder Hackathon with an LLM-judged supply-chain malware detector. BSc (Hons) Computer Science, 2026.

SKILLS

Machine Learning & Computer Vision: PyTorch, scikit-learn, OpenCV, Albumentations, ordinal regression (CORAL), model calibration (isotonic regression, temperature scaling), Grad-CAM, evaluation design (bootstrap confidence intervals, sensitivity/specificity operating points), data leakage prevention, feature engineering
LLM Systems: Anthropic API, OpenAI-compatible APIs (Featherless, Cerebras, Groq), Ollama local inference, LLM-as-judge, structured model output, multi-provider fallback and routing, autonomous agents, prompt engineering
Backend & Infrastructure: Python, FastAPI, REST APIs, SQL, SQLite, sandboxed execution (Daytona), strace, Linux, Bash, Git/GitHub
Testing & Reliability: pytest, unit testing, append-only audit logging, structured logging, error handling, reproducible configuration
Data: pandas, NumPy, Matplotlib, Seaborn, relational schema design (3NF), multi-table SQL
Programming Languages: Python, SQL, C/C++, Bash, HTML/CSS/JavaScript

PROJECTS

Diabetic Retinopathy Screening System

Solo project | 2026 | github.com/yc82628/Medical_Diagnosis_w_DL
Stack: Python, PyTorch, OpenCV, Albumentations, FastAPI, SQLite, pytest

- Designed and built an end-to-end retinal image grading system that fails safely: classical quality and anatomy gates run before the model, so blurred or non-retina images go to manual review instead of being graded as healthy.
- Used an ordinal CORAL head instead of a 5-way softmax so a severe misgrade (grade 4 read as 0) costs far more than an adjacent one; trained at 512 px after measuring that a 10 px microaneurysm shrinks to 1.3 px at 224 px.
- Found and fixed data leakage: a naive row-level split placed 142 of 400 patients in both train and test. Built unit-tested, patient-grouped stratified splits and patient-level bootstrap CIs (15% wider, and more honest, than image-level).
- Selected calibration empirically: isotonic regression cut expected calibration error from 0.102 to 0.031, while temperature scaling made it worse (0.116). Model selection uses specificity at 90% sensitivity, not accuracy.
- Shipped a FastAPI grading service: abstention band that defers borderline cases to human graders, clinical-priority worklist, append-only SQLite audit log (model version and preprocessing fingerprint, image hashes only), and Grad-CAM with a randomised-weight sanity check to flag shortcut learning.
- Status: pipeline runs end to end on synthetic data; external validation on Messidor-2 is the next step.

DayTonator: AI Supply-Chain Malware Detector

Top 10, AI Builder Hackathon (Daytona) | Team project | June 2026 | github.com/yc82628/Daytonator
Stack: Python, FastAPI, Daytona, strace, Featherless, Anthropic API

- Detects "slopsquatting": malware registered under package names that AI coding agents hallucinate, including payloads that stay dormant in a normal sandbox scan.
- Owned the sandbox-forking layer: snapshot one clean Daytona sandbox and fork it into 5 isolated environments (control, SSH key, AWS credentials, production-like host, clock jump) so every detonation starts from an identical state.
- Each fork records file, network and process activity with strace; behaviour seen in a baited fork but not in the control is provably bait-triggered, exposing evasive malware that a single clean sandbox records as zero events.
- An LLM judge (Qwen2.5-72B via Featherless, then Anthropic) turns the cross-fork evidence into a scored safe / suspicious / malicious verdict with a plain-English rationale, with a deterministic heuristic as final fallback.

Midknight Watcher: Autonomous Coding Agent
42 Berlin x Needle Hackathon | Team of 2 | May 2026 | github.com/yc82628/Ai-Agent-
Stack: Python, Cerebras, Groq, Ollama, Git

- Agent reads a hidden spec, then plans, writes, tests and repairs its own code in a ~620-line custom loop (no framework, so it could be patched live), with cross-family model fallback: Qwen3-235B, Llama 3.3 70B, then local Qwen3.5 9B.
- Operated the overnight autonomous run and led the recovery: traced 0/150 overnight scores to file timestamps confusing the test runner rather than to the agent's code, and restored the 92/150 build after an iteration-30 infinite-loop regression using per-iteration commits.
- Result: 92 of 150 public tests (61.3%) and 30 of 100 hidden tests passed, with all solution code written by the agent.

EXPERIENCE

Working Student, Mmaah Korean Restaurant
Berlin | Mar 2025 - Nov 2025

- Introduced a delivery checklist system that cut order discrepancies by 15%.
- Maintained inventory records across 50+ product categories in a high-volume service environment.

EDUCATION

BSc (Hons) Computer Science, BSBI Berlin School of Business and Innovation
Berlin | Oct 2023 - Oct 2026
Coursework: Data Structures & Algorithms, Software Development, Database Systems, Object-Oriented Programming
Academic projects: Stock price forecasting pipeline (Python, scikit-learn, pandas; feature engineering improved RMSE by 12%). Healthcare relational database in 3NF with a Python interface and multi-table SQL queries.

LANGUAGES

English (C1) | Mandarin (B1) | German (A2)